Reasoning About Distributed-Systems and Security Trade-offs in Choir: A Field Guide

Part 6 of the Choir learning series, written to teach rather than to decide. 14 October 2026. Repository: `origin/main` at 980ffaf4e. This guide reads code and the problem records in `docs/problems/`; it had no live host access, and it did not contact staging or Node B. Mutation class of this document: **green** (docs only).

Evidence labels, as in the isolation review: **code** (read in the repository at the commit above, with a path), **doc** (an existing Choir record, usually a problem record that itself cites live traces), **upstream** (a published idea or paper from outside Choir, named so you can look it up), and **inference** (reasoning that has not been tested; treat it as a hypothesis).

How to read this

The other five parts of the series each teach one subsystem. This one teaches the habits of mind that cut across all of them. It is organised as eleven principles. Each has the same four parts:

1. **The principle**, stated in one line and taught from first principles, with the classic outside example that made engineers name it.
2. **A Choir incident** that shows it, read from its problem record and from the code of its fix, with paths and commits.
3. **The question to ask next time**: one reusable checklist line.
4. **The trade-off**: when the opposite choice is the right one. A principle without its counter-case is a slogan.

Section 12 is a one-page pre-flight card that compresses all eleven into questions you can ask of any design change. Read it last, then keep it.

The incidents are real and recent; most are from the week of 10 to 14 October 2026. They were picked because each one was found, documented before it was fixed (CLAUDE.md "Problem Documentation First"), and then fixed in code you can read. Where a fix is on a branch or not yet built, the text says so.

The short answer

1. **Most failures are not crashes.** A process that is alive, accepts connections, and never answers (a *gray failure*) did more damage to Choir this month than any clean crash: a deadlocked vCPU, a hung guest that made vmctl's own health slow, a model run that took 38 minutes. Design for "slow and silent", not just "dead".
2. **Every wait needs a bound, and the bounds must nest.** A caller's timeout must be longer than everything it waits on, or work succeeds after the caller gave up. Choir's guest search and gateway search both used 30 seconds; 10% of searches finished at the gateway after the guest had reported failure.
3. **Exactly-once is an illusion you build from at-least-once delivery and idempotent handlers.** Choir's best code does this with idempotency keys and journals (the updater's operation journal, the inbox command replay, Texture's per-cell revision identity). Its worst incident in this class was the reverse: a cursor that acknowledged records before they were handled, so 28 captures were silently lost.
4. **One writer per fact.** When two places can say different things about the same fact, the system will eventually publish the wrong one. A management brief became 15 front-page headlines because the story title had a writer that was not Texture.
5. **Fail closed for security, fail open for availability, and decide which one you are building on purpose.** Choir's host firewall failed open (FORWARD policy ACCEPT); its planned-restart marker fails closed (any doubt means "crash"); its memory admission fails open on a missing sample. All three are choices; only the first was accidental.

6. **Size the blast radius before you need it.** A circuit breaker keyed by provider let one free model's rate limit take down the paid model beside it. A deploy's stop ordering let one service's 11-minute drain take the whole site down.
7. **Assume each layer is breached.** Firecracker runs as root without its jailer, so one VMM escape is every tenant. Defence in depth means the next layer is sized for the failure of the one in front of it.
8. **A crash restart ends work; a planned restart may resume it once.** This rule exists because auto-resume after a crash is how crash loops happen. The planned-restart marker is consumed before resuming, so a resume that crashes cannot loop.
9. **A serial queue with arrivals faster than service grows without bound.** Management's one-turn-at-a-time design stopped the paper for 30 minutes; at 28 sources even one task at a time could not keep up. Backpressure, bounded concurrency and admission control are the tools.
10. **Read the trace before you explain behaviour.** A desk that "chose not to" do something, and a desk whose correct act committed late, look identical from outside. The CPU of the paper turned out to be index-less store scans, not agent work, and only a profile showed it.
11. **Correlate every answer with its question.** Two concurrency bugs in one week came from code that assumed the next frame on a connection, or the current value of a field, belonged to it.

1. How distributed systems fail

1.1 Primer

A distributed system is several processes that cooperate over a network or a shared medium and can fail independently. The last word is the whole problem: one part fails while the others keep running and keep acting on what they last believed. Engineers classify failures by what the failed part does (upstream: the taxonomy in Lamport, Shostak and Pease, "The Byzantine Generals Problem", 1982, and in every distributed-systems textbook since):

failure mode	what the part does	how others see it
crash (fail-stop)	stops and stays stopped	connections refused; no answers
omission	drops some messages, sends or receives	some requests never answered
timing (performance)	answers, but too late	answers that arrive after the caller moved on
Byzantine (arbitrary)	answers wrongly, possibly plausibly	answers that look right and are not

Crash is the easy case: it is detectable (eventually) and it does not lie. Most real outages are in the other three rows. Microsoft's paper "Gray Failure: The Achilles' Heel of Cloud-Scale Systems" (Huang et al., HotOS 2017, upstream) named the most common shape: a component that the failure detector considers healthy while its users experience it as broken. A disk that answers health checks but takes 30 seconds per write. A process that accepts TCP and never sends a byte. The phrase "slow is the new down" is the practitioner's version: in a system of timeouts, a slow part is indistinguishable from a dead one, except that it holds resources while it is slow.

The fourth row matters more in Choir than in most systems, because Choir has a component that is Byzantine by nature: **the model**. A language model's output is plausible by construction and correct only sometimes. It does not lie on purpose, but from a systems view it is an arbitrary-failure component, and every boundary where its output becomes state must treat it the way a security engineer treats input from the internet.

1.2 Choir incident: alive, accepting, and frozen

On 10 October the owner's computer stopped answering at 15:51 UTC (firecracker-sigpipe-deadlock-after-vmctl-restart, doc). The Firecracker process was alive. The guest kernel accepted TCP on its service port. Every HTTP path hung, even a 404. The stack of vCPU 0, read with `eu-stack`, showed the cause: the guest wrote to its serial console; Firecracker wrote that to its stdout, which was a pipe whose reader was a goroutine inside the vmctl process that had launched it; that vmctl had been replaced at a deploy, so the pipe had no reader; `write(2)` raised SIGPIPE; Firecracker's SIGPIPE handler tried to log, took a logger lock it already held, and deadlocked the vCPU thread while that thread held the serial device's lock. One vCPU never ran again.

Three failure modes stack in that story:

- **Hidden fate-sharing** turned a deploy (replacing vmctl) into a latent fault in every guest vmctl had launched. The guest's console depended on a process the guest had no reason to know about.
- **Gray failure** hid it. The wedge watchdog acts only on a guest that refuses connections; this one accepted them.
- **Slow spread into the platform.** A sibling record, vmctl-health-fate-shares-a-hung-guest (doc), shows the same hung guest making vmctl's own `/health` take 8 seconds, because `HandleHealth` called `guestBusy()`, which probed every guest's `/health` in turn with a 4-second timeout. The proxy probes vmctl with a 2-second timeout, so `choir.news/health` reported the host degraded and the deploy gate failed. One frozen guest made the platform look down.

The fix removed the hidden dependency instead of detecting it better. `startFirecrackerProcess` (`internal/vmmanager/manager.go`, code) now opens a console *file* and hands it to Firecracker as stdout (`cmd.Stdout = console`), with the comment "The console is a file firecracker writes directly, never a pipe that needs this process to read it". The record names the deploy as 86770ae7.

1.3 Choir incident: the model as a Byzantine component

On 14 October, 15 of the 100 titles on the live front page were desk instructions such as "Write the story of Switzerland proposing to ban profiling-based advertising aimed at minors: what the Swiss federal government has actually published…" (instruction-text-published-as-story-titles, doc). The management prompts already said the Cast's first argument "is the story's working title". The model sometimes wrote a headline and sometimes wrote a brief. Nothing downstream checked. Section 5 returns to the structural cause; the point here is that a prompt rule is not enforcement. The same record family shows `choir.Feed` accepting a capture id that named no capture (a mangled id, in captures-undisposed-after-failed-management-runs, doc, "Unvalidated capture ids"). A text search of `internal/store`, `internal/agentcore` and `internal/yaegikernel` for a refusal of an unknown capture id found none (code; *inference* that this is still open).

1.4 The question to ask next time

> *If this component is alive but silent, or answers plausibly and wrongly, who notices, how long does it take, and what does it hold while it waits?*

1.5 The trade-off

Treating every component as potentially Byzantine is expensive: you validate, cross-check and sometimes run things twice. Inside one process, between functions you wrote and tested, it is waste; trust the type system. The line belongs at **trust boundaries**: process boundaries, the network, and every place model output becomes durable state or reader-facing text. Choir draws it in the right place when it validates a model-written act at the store (an idempotency digest, a compare-and-set on a head, a publish gate) and in the wrong place when it relies on a prompt sentence.

2. Timeouts and deadline budgets

2.1 Primer

A timeout converts a timing failure into a crash failure you can handle. Without one, a caller waiting on a slow part becomes slow itself, and the slowness climbs the call stack until something with a timeout (a user, a load balancer) gives up on everything at once.

Two rules make timeouts work in a chain of calls:

1. **Nest them.** If A calls B calls C, then A's timeout must exceed B's whole budget (including B's retries and fallbacks), and B's must exceed C's. If they are equal or inverted, C finishes after A gave up: the work is done, paid for and thrown away, and A reports a failure that did not happen.
2. **Propagate deadlines, not durations.** A fixed "30 seconds" at each layer does not know how much of the caller's time is left. A deadline (an absolute time) passed down with the request lets each layer stop when the caller stops. Go's `context.Context` exists for this; gRPC propagates deadlines across the network for the same reason (upstream).

Jeff Dean and Luiz Barroso's "The Tail at Scale" (CACM 2013, upstream) adds the third lesson: when a request fans out to many parts, the slowest part sets the latency, so the tail (p99), not the median, is what you design for. Their remedies, hedged requests and tied requests, trade extra load for shorter tails.

2.2 Choir incident: the guest gave up just before the gateway succeeded

In the 11 October demo, research reported that a search "timed out at the search gateway, so no source-bearing report was added", and Texture correctly rejected a report with no sources (guest-search-timeout-equals-gateway-provider-timeout, doc). The gateway's log for that query said it succeeded, 30 seconds and a few hundred milliseconds after the request. The record found three equal budgets: the guest's gateway search client at 30 s, each provider's HTTP client at 30 s, and the gateway handler's `context.WithTimeout` at 30 s. A later measurement over 509 searches found 51 (10%) that ran to exactly 30.0 s at the gateway and were logged as succeeded; every guest-side timeout in two saved traces matched one of them. The desk then explained its thin coverage as "search-provider cooldowns and timeouts". A system that cannot tell slow from failed teaches its users the wrong lesson.

The first designed fix was a deadline ladder (provider about 10 s, gateway about 25 s, guest about 35 s). The owner chose a different shape: make search asynchronous, so nothing waits on the slowest provider at all (re-engineering decisions §5, doc). Commit 88734e8ab (code) added `choir.SearchStart`, which returns a stub at once; each provider's answer lands in the desk's REPL state as a `StreamSearchArrival` frame. Today the guest's search client is built with `hostlink.NewClient(0)` (`internal/search/search.go`, code: no per-attempt bound on the stream), and the gateway bounds the stream with `h.searchClient.StreamBound()` (`internal/gateway/handlers.go` and `internal/gateway/search_plane.go`, code), derived from the plane's request timeout and escalation window plus a margin.

2.3 Choir incident: a cell with no bound

The management serial-turn record (management-serial-turn-blocks-paper, doc) found a management cell that ran 860,565 ms (14 minutes) and ended `frame: read header: EOF`: its worker died, and nothing timed it out. A cell had a deadline only when the model set `timeout_ms`. The fix (f10b655a4, code) is `managementCellDeadline = 300 * time.Second` in `internal/agentcore/tools_desk.go`. Its comment derives the number from measurement: max(300 s, 2 × the p99 management cell duration measured on the live paper), where p99 was 4.6 s over 295 cells. Note two details worth copying:

- **The bound is derived from data, with a floor.** It is not a guess, and it leaves a large margin over normal behaviour so that it only catches pathology.
- **The override fails safe.** `CHOIR_MANAGEMENT_CELL_DEADLINE` accepts a Go duration; "a value that does not parse, or a negative one, keeps the default, so a typo cannot unbound the cell" (code comment).

Management tasks also bound each model call (`managementTaskCallTimeout = 3 * time.Minute`, `internal/agentcore/management_tasks.go`, code), "so one slow call cannot use the task's whole deadline". That is rule 1 in miniature: the inner bound is shorter than the outer one.

2.4 The question to ask next time

> *Write down every timeout on this path, from the user to the last provider. Does each one exceed the whole budget of what it waits on, including retries and fallbacks?*

2.5 The trade-off

Short timeouts fail fast and free resources, but they turn a slow success into a failure and invite retries that add load exactly when a dependency is struggling. Long timeouts tolerate slow parts but tie up workers and hide gray failure. Some waits should have no fixed timeout at all: Choir deliberately gives research no host cell deadline, because `choir.SearchWait` can legitimately block for minutes (`deskCellHostDeadline`, `internal/agentcore/tools_desk.go`, code: "other desks wait on legitimate long calls"). When the wait is legitimate and long, the better tool is often to stop waiting synchronously: make it asynchronous and let the caller poll or be notified, which is exactly what async search did.

3. Correlation, ordering and races

3.1 Primer

When several requests share one connection, one queue or one variable, each response must be tied to the request that caused it. Network protocols do this with request ids (HTTP/2 stream ids, JSON-RPC `id`, TCP sequence numbers). Shared memory does it with synchronization: a lock, an atomic, or a single owner goroutine. Code that reads "the next frame" or "the current value" and assumes it is its own works only while nobody else is reading or writing. The classic failure is the *mis-correlated reply*. The best-known modern case is HTTP "response queue poisoning" (upstream: James Kettle's request-smuggling research, PortSwigger, 2019 to 2020): when a front-end and a back-end disagree about where one request on a shared keep-alive connection ends, every later response goes to the wrong user. The lesson is that every answer must carry something that ties it to its question.

3.2 Choir incident: parallel calls got each other's answers

On 12 October a probe ran four goroutines calling `choir.*` network verbs from one desk cell (parallel-choir-calls-cross-wire-results, doc). Caller `a` got `c`'s answer, `b` got `a`'s, `c` got `b`'s, every run. `egressRoundTrip` wrote a request frame and then read the next frame on the connection; it never matched the response to the request id it had just minted, and nothing serialized write and read. The research prompt itself recommended "parallelism appropriate to the task". Any research run that followed that advice could have attributed search results to the wrong query with nothing in the trace to show it.

The fix (cd2a224f5, code) is `workerMux` in `internal/yaegikernel/worker_mux.go`: "One reader loop reads every frame: broker results go to their waiter by request id, and cell frames go to the session loop in order." It also closes the second-order case: a goroutine that calls a verb after its cell returned gets `errCallAfterCell` instead of being served under the next cell's identity, and `cellMu` orders calls against the cell boundary so every request a cell sends is on the wire before its result frame.

3.3 Choir incident: a plain field swapped while read

CI run 38476951550 caught a data race (post-commit-observer-race, doc): a test swapped the appender's post-commit observer (a plain function field) while a background append read it. Production wrote it once at start, so the race was test-only, but any later runtime swap would race the same way. The fix (19e56c54e, code) holds it in `atomic.Pointer[func(kind EventKind)]` (`internal/computerevent/appender.go`). Small, but notice the reasoning in the record: the fact that production "happens" not to race today was not accepted as safety.

3.4 The question to ask next time

> *If two of these run at once, how does each one know the answer it reads is its own?*

3.5 The trade-off

Correlation and locking cost complexity and sometimes throughput. A single lock around write-then-read is correct and serial; a request-id router is concurrent and needs its own failure handling (what happens to waiters when the reader dies? `workerMux.readLoop` closes them all). Sometimes the best answer is to remove the concurrency from the untrusted side: the problem record's second hypothesis was that model-written goroutines should not call network verbs at all, and async search moved the fan-out to the host. Choose where concurrency lives as deliberately as how it is synchronized.

4. Idempotency and the exactly-once illusion

4.1 Primer

Over an unreliable channel you cannot know whether a message you sent and got no answer for was received. This is the Two Generals Problem (upstream): no finite exchange of acknowledgements makes both sides certain. So real systems choose a delivery guarantee:

- **at-most-once**: send once, never retry. Nothing is duplicated; some things are lost.
- **at-least-once**: retry until acknowledged. Nothing is lost; some things are duplicated.
- **exactly-once**: does not exist on the wire. It is *built* as at-least-once delivery plus an **idempotent** handler: one that produces the same outcome no matter how many times the same request arrives.

The standard tool is an **idempotency key**: the caller names the operation once (a key that survives retries), and the handler records the outcome under that key; a repeat returns the recorded outcome instead of acting again. Stripe's API popularised this for payments (upstream, "Idempotent requests" in Stripe's API docs). The subtle partner rule is about **acknowledgement**: you may only acknowledge (advance a cursor, delete a queue message, mark as read) *after* the work is durably done, or in the same transaction. Acknowledge first and you have built at-most-once by accident.

4.2 How Choir does it well

Three places in Choir are worth studying as patterns (all code):

1. **The updater's operation journal.** `Updater.Apply` (`internal/updater/updater.go`) opens `operations/<idempotency key>.json` before anything else. If the journal has an outcome, it returns that outcome and does nothing. It also checks a *request commitment* (a hash of the request): the same key with a different request is refused with `ErrIdempotencyConflict`, so a key cannot be reused for a different operation. Pre-mutation refusals are journaled too, "so the caller can tell them apart from mutated-then-unrestored failures". The agent side names the key with `platformUpdateIdempotencyKey("apply", offer.UpdateID)` (`internal/agentcore/platform_update.go`).
2. **The inbox command replay.** `Store.CommitInboxRecords` (`internal/store/inbox.go`) first calls `replayInboxCommand` with the request's `CommandID` and `CommandDigest`. A command already committed returns its receipt. If the conditional batch write then conflicts (`objectgraph.ErrConflict`), it checks replay again before reporting `ErrConcurrentStateChange`: the conflict might be its own earlier attempt.
3. **Texture's per-cell revision identity.** `CommitCellTextureAuthor` (`internal/textureowner/texture_cell_author.go`) receives "the deterministic per-cell idempotency identity the reducer derives, so a replayed cell replays the same commit". It looks for the receipt first, because `ApplyTextureTurn` is head-pinned and "a second dispatch would CAS-conflict on the head the first commit advanced".

The pattern in all three: **name the operation before doing it, record the outcome under the name, and check the record first.**

4.3 Choir incident: acknowledged before handled

On 14 October, 28 captures from an 08:22 burst on the live paper were never disposed (captures-undisposed-after-failed-management-runs, doc). The trace: run a8081845's first cell looped over `choir.Records()` and printed records 508 to 514. It staged no acts. The run then failed on an open circuit. The next run was shown only record 515. Records 508 to 514 were behind the cursor.

The mechanism, from code: `commitRecordsRoute` committed each cell's records **and** a cursor advance over what the run had been shown. A cell that only read still committed the cursor advance alone. So reading was acknowledging. The run failed after the acknowledgement and before the work: classic at-most-once by accident.

The fix (e3437f1cd, code) moves the acknowledgement to the end of the turn. `markTurnRecordsRead` (`internal/agentcore/inbox_records.go`) marks read what a management turn was shown only when the turn **completed or parked**; a failed or cancelled turn marks nothing. A cell that acts still commits its acts and the read mark together, in one transaction. Note three further details of the fix:

- The read mark is itself idempotent and guarded: its `CommandID` is `turn-records-read:<run>:<seq>`, and the cursor advance carries `ExpectedSeq` (a compare-and-set on the cursor).
- A failed turn schedules one durable retry wake with exponential backoff (`scheduleFailedTurnRetry`: 2, 4, 8, 16 minutes, then stop), so a provider outage cannot become a hot loop.
- The record keeps its **remaining error field** honest: a run that disposes some captures in one cell and then fails still marks the rest read. Fixing that needs a per-capture disposition index. S2b added a capture disposition index ("one capture, one disposition", `internal/agentcore/management_tasks.go` comment, code) for concurrency; whether it closes this residual was not checked here (*inference*: it is the right shape for it).

4.4 The question to ask next time

> *If this message is delivered twice, what happens? If the process dies right after it acknowledges, what is lost?*

4.5 The trade-off

Idempotency keys cost storage (journals, replay rows) and design effort (what exactly is "the same request"?). At-most-once is the right choice when duplication is worse than loss and the operation is cheap to redo by hand: a log line, a metric, a notification nobody depends on. Choir's search arrivals are an example of benign loss: a missed arrival costs a little evidence, not correctness. For anything that moves money, mutates a computer, or carries an obligation (a capture that must get a disposition), build at-least-once plus idempotent handling and pay for it.

5. Single state authority

5.1 Primer

Every fact in a system should have **one writer**, and every other copy should be a derived view that can be rebuilt from it. The failure this prevents is called the *dual-write problem* (upstream; the standard remedies are the transactional outbox and change-data-capture): when code writes the same fact to two places (a database and a cache, a database and a message bus), any crash between the writes leaves them disagreeing, and readers of each see a different world. Martin Thompson's *single-writer principle* (upstream, from the LMAX work) says the same thing for concurrency: a datum written by one thread needs no lock.

Choir's standing questions make this question 5: "What is the single authority for this state? If two projections can disagree about a fact, name which one wins before touching either. If none wins, that is the bug." Its receipt is the processor lifecycle split across five projections (run state, trajectory, processor-resolution, a retired ledger, an

admission counter), where "each pairwise disagreement froze the pipeline a different way". The desk ontology carries the same principle at the product level: **Texture is the sole writer of canonical document revisions** (CLAUDE.md, Desk Ontology).

5.2 Choir incident: a title with the wrong writer

The 15 instruction titles (section 1.3) had a structural cause, traced in the problem record (doc) and confirmed in code:

```
choir.Cast("texture:new", objective, spec)       management writes the brief
  -> CommitmentRecord.Directive.Objective        (Texture reads it as its brief)
  -> store.planStoryOpenings / planStoryObjects   Document.Title = objective
  -> Texture revisions commit                     Title never touched
  -> AutoPublishRevision -> publish request       Title = doc.Title
  -> front page row                               the brief, as a headline
```

One string did two jobs (work brief and story title), and the title was written once, by management, and never again. Texture was the sole writer of the story's *body*, and the body led with a real headline in 9 of the 10 affected stories, but nothing Texture wrote reached the title. The rule "Texture is the sole writer" held for revisions and was quietly false for the story's most visible field.

The fix (201fe33ca, code) gives the title one writer. `storyTitleAfterRevision` (`internal/store/story_open.go`) sets a story's title from the revision's headline, which `texturedoc.Headline` (`internal/texturedoc/headline.go`) reads from the body's first block when that block is a heading. It happens "in the same object write that moves the head, so title and head cannot disagree" (problem record). The opening objective survives only as a working title until Texture's first headline. And `AutoPublishRevision` (`internal/agentcore/story_publish.go`) holds a story revision that does not lead with a headline, the same way it holds one that cites no source (`texturewrite.CitesSource`).

5.3 How the store enforces one writer

Two code patterns carry single authority in Choir:

- **Compare-and-set on the head.** `ApplyTextureTurn` (`internal/store/texture_turn.go`, code) requires `ExpectedHeadRevisionID` and refuses a turn when the document's `CurrentRevisionID` is not that head. Two writers cannot both advance the same head; the second gets a conflict and must re-read. This is optimistic concurrency control (upstream), and it is how a single *logical* writer is enforced even when there are several processes.
- **One write gate.** Commit 7f287a135 (code, H-OG1) found that the object-graph kind registry was consulted only by `objectgraph.Service`, while `DoltStore` and `platform.ObjectGraphStore` wrote any kind; 26 object kinds and 12 edge kinds had reached the tables unregistered. The fix put one function, `objectgraph.CheckWrite`, first in every mutation method of both durable stores. A registry that only some writers consult is not an authority.

5.4 A related gap: dispatch outside the ledger

inter-agent-communication-bypasses-precommitment-records (doc) counted, over 1,286 run event streams, that every dispatch Texture made (283 uses) travelled as a lifecycle control outside the commitment ledger, while only the answers came back as records: "The ledger is a one-way report channel." When the ledger is meant to be the single authority for who asked whom to do what, a side channel is a second authority. Later work moved acts onto records (see part 2); this guide did not audit how much of the side channel remains.

5.5 The question to ask next time

> *Who is the one writer of this fact, and can any other code path change it or a copy of it?*

5.6 The trade-off

A single writer is a bottleneck by design. Texture as the sole writer of a story means a story's revisions are serial; management's tasks needed a "story-library head" (a compare-and-set on the library) so that two concurrent tasks cannot open the same story (`internal/agentcore/management_tasks.go` comment, code). When throughput matters more than a single serial history, you partition the fact (one writer per story, not one per paper) or accept a merge rule (last writer wins, or a CRDT, upstream) and the anomalies that come with it. What you must never do is have two writers *by accident*.

6. Fail closed or fail open

6.1 Primer

When a check cannot decide (the dependency is down, the input is unreadable, the sample is missing), the code must still do something. It either **fails closed** (deny, refuse, stop) or **fails open** (allow, proceed, keep running). Building codes make the trade-off concrete: an electronic lock on a fire exit must be *fail-safe* (unlock on power loss, so people get out), while a lock on a vault is *fail-secure* (stay locked) (upstream). Neither is right in general. Security controls usually fail closed, because the cost of a wrong "yes" is a breach. Availability controls often fail open, because the cost of a wrong "no" is an outage.

A **circuit breaker** (upstream: Michael Nygard, *Release It!*, 2007) is a deliberate fail-closed device for availability. After N failures it stops calling a dependency for a while and fails fast, which protects the dependency from load and the caller from waiting. Its correctness depends entirely on two choices: **what counts as a failure**, and **what the breaker is keyed by**. Get either wrong and the breaker itself becomes the outage.

6.2 Choir incident: a breaker that counted the caller's mistakes

On 10 October, nine `400 Bad Request` responses for one computer's engineering run opened the `opencode-go` breaker, and calls from a different computer failed with "circuit open (upstream unhealthy)" (inference-breaker-trips-on-client-errors, doc). A 400 is the provider refusing *our* request: evidence about the caller, not the provider. Counting it let one computer's bad requests fail closed for everyone. A sibling, `search-plane-cooldown-on-caller-cancel`, had already counted caller cancellations as provider failures.

The fix (589a68bc6, code) is `providerFailureCounts` in `internal/gateway/circuit_breaker.go`: a caller cancellation does not count, and a 4xx other than 408 and 429 does not count, with the comment "counting them let one computer's bad requests open the provider for every computer". A deadline overrun still counts: "the provider was slow".

6.3 Choir incident: fail-open where it should be closed

The isolation work found the host firewall failing open (tap-deny-rule-covers-one-of-sixteen-guest-subnets, doc and code). The guest address pool spans sixteen /16 networks (`vmSubnetSecondOctetBase = 200`, `vmSubnetSecondOctetCount = 16` in `internal/vmmanager/manager.go`), but `ensureTapIsolationRules` denies forwarding only to `10.200.0.0/16` (code, same file). Live reads on Node B found every current guest inside 10.200, so the gap was latent, and stale rules showed earlier guests had used 10.202. The record's sharpest line: "The FORWARD policy is ACCEPT, so isolation rests entirely on per-tap rules. A tap whose rules were never installed, or were lost, can forward anywhere." A fail-open default plus one wrong constant is a cross-tenant path. The fix, a default-deny guest chain over the whole pool with teardown on every exit and an orphan sweep, is on branch `fix/tap-deny-guest-range` (816077e83) and is not on main (code: main still has the 10.200.0.0/16 rule).

6.4 Choir's deliberate choices, both ways

Reading the code with this lens shows several decisions made on purpose (all code):

site	when it cannot decide	posture	why
`ConsumePlannedRestartMarker` (`internal/agentcore/planned_restart.go`)	marker unreadable, unparsable, stale, wrong epoch, or cannot be removed	closed: treat the boot as a crash	a wrong "planned" resumes work and can loop; a wrong "crash" only closes work visibly
`settlePendingPlatformUpdateAfterCrash` (`internal/agentcore/platform_update.go`)	updater journal mid-flight	closed: leave the update pending, refuse later updates until an owner act	"the effective state is unknown, and refusing later updates until an owner act is fail-closed"
`admitHostMemoryLocked` (`internal/vmctl/host_capacity.go`)	memory sample missing	open: admit	"unknown; never refuse on a missing sample": a broken sampler must not stop every boot
same function	computer is protected (owner, premium)	open: always admit	the owner's computer must start even under pressure
`AutoPublishRevision` (`internal/agentcore/story_publish.go`)	citations cannot be read	closed: hold the revision	a wrong publish reaches readers; a held revision is a visible gap

6.5 Choir incident: fail-safe with no bound becomes a wedge

The most instructive case is a fail-closed rule that was right and still caused harm. Phase 0 of vmctl's lifecycle work made "uncertainty never destroys a computer": when vmctl cannot tell whether a guest is healthy, it refuses to reap it (owner-computer-stranded-after-vmctl-restart, doc). On 10 October the owner's computer was alive, unanswering (it did not even answer ARP), protected, and impossible to restart: every automatic path refused by design, and the owner's restart button only "resolved", which hit the same refusal. "Nothing bounds the protection, so it holds forever." The owner's ruling: "Users shouldn't ever have to do this. It should all be automated." The bound became evidence-based: a guest that refuses TCP to its service port for five minutes is *wedged*, and reconcile stops it with a receipt. In code, `guestWedged` (`internal/vmctl/wedge.go`) records one network probe per sweep and reports a guest that "has refused connections for the whole wedge window", and `Wedged` is counted in `VMProcessReconcileResult` (`internal/vmctl/host_capacity.go`).

The lesson: **a fail-closed rule needs an exit condition**, stated in evidence, or "safe" becomes "stuck".

6.6 The question to ask next time

> *When this check cannot decide, does it allow or deny, was that chosen, and what bounds how long it stays in that state?*

6.7 The trade-off

Fail closed protects integrity and secrecy at the cost of availability; fail open protects availability at the cost of integrity. Choose by asking which wrong answer is worse *and who pays for it*. For a breaker, also ask whose failure it is: a breaker should open only on evidence about the thing it guards. For a guard on a destructive act, pair the refusal with a bound and an escape that does not require a human at 3 a.m.

7. Blast radius and bulkheads

7.1 Primer

Blast radius is what fails when one thing fails. **Bulkheads** are the walls that limit it, named after the watertight compartments in a ship's hull: a breach floods one compartment, not the ship (upstream; the *Titanic* is the usual cautionary tale, because its bulkheads did not reach high enough and water spilled over the top). In software, bulkheads are separate pools, separate processes, separate cgroups, separate breakers, separate hosts. AWS's "cell-based architecture" and "shuffle sharding" (upstream, Amazon Builders' Library) are the same idea at fleet scale.

Choir's standing question 7 is the read-path version: "What does this read path fate-share with? Enumerate every component that must be simultaneously healthy for the read to succeed." Its receipt: the stories API once traversed auth, proxy, vmctl, VM boot, guest runtime and a single-connection Dolt, so "every deploy closed the publication window".

7.2 Choir incident: one model's rate limit took down its neighbour

On the morning of 14 October, some published stories were one-paragraph stubs (free-model-429-opens-provider-circuit-stub-stories, doc). A free preview model, `cat-jam-free`, was returning 429 (rate limited). Its failures opened the breaker for the whole `opencode-go` provider, which also served `deepseek-v4.1-flash`, a paid model that was healthy upstream (the owner's provider console showed DeepSeek answering 200 at the same minutes). The breaker's key was too coarse: its blast radius was every model on the provider. Research runs failed, Texture wrote revisions saying the claims were unverified, and auto-publish published them.

The record shows a second coupling that amplified the first: retries. Each runtime call was up to three gateway calls (the gateway client retried a 502), and the tool loop retried a "429" text three more times at 5, 20 and 60 s. So one rate-limited thread alone produced the five failures that opened the breaker. By arithmetic on those documented counts, one logical call could make up to about twelve upstream attempts (*inference*: an upper bound, not a measurement). Retries multiply load at every layer that has them; stack three layers and a small fault becomes a storm.

The fixes (all on main, code):

- **Smaller compartments.** bde0a169f keys breakers by (provider, model) plus one provider breaker that counts only transport failures to reach the provider at all (`providerUnreachable`): "A provider serves several models, and most failures belong to one of them" (comment on `CircuitBreakingProvider`).
- **Failover that walks.** `RunToolLoop` (`internal/toolregistry/toolloop.go`) now walks the remaining pool members, each at most once, instead of only `fallbackLLMConfigs[0]` (which was on the same provider behind the same open breaker).
- **A gate on the reader-facing surface.** `AutoPublishRevision` holds a revision that cites no source.

7.3 Choir incident: one service's drain took down the site

Every deploy on 13 to 14 October took `choir.news` down for 3 to 11 minutes (deploy-takes-staging-down-during-gateway-drain, doc). NixOS's switch stops every changed unit, then starts them only after all the stops finish. The gateway's stop is a graceful drain of in-flight model calls (`TimeoutStopSec = "11min"` in `nix/node-b.nix`, code). Proxy, auth and corpusd stopped in a second and then waited for the gateway. A drain designed to protect one service's requests took every other service down with it. The fix (2db4bc02, code and commit message) gives auth, maild, corpusd, gateway and proxy `restartIfChanged = false`, and the CI deploy restarts auth, maild, corpusd and proxy together, then the gateway "last and alone". Proven on a later deploy with 417 of 417 health probes at 200 (doc).

7.4 Choir incident: one host, no admission

On 10 October Node B ran out of memory (node-b-oom-from-retained-qa-computers, doc). Disposable QA computers were never stopped; each held 2.5 to 3.8 GiB. Pressure reclaim saw the pressure ("pressure=true" 35 times) but its registry held only two computers; ten running VMs had dropped out of the registry across vmctl restarts. The kernel's OOM killer then chose among host processes, including vmctl itself (which died with SIGBUS). The QA probes and the owner's computer shared one host with no wall between them. The fix (cf0969cf per the record; code in `internal/vmctl/host_capacity.go`) added three bulkheads: `ReconcileVMProcesses` makes the registry account for every Firecracker process; memory admission refuses a new unprotected computer with 503 and Retry-After below the floor; and OOM priority by class (vmctl -900, protected VMs -400, others +500) tells the kernel which compartment to flood first.

7.5 The question to ask next time

> *If this one thing fails or slows, list everything else that fails with it. Is each item on that list there on purpose?*

7.6 The trade-off

Bulkheads cost utilisation. Separate pools sit idle while their neighbour is overloaded; separate breakers each need traffic to learn health (a model breaker on a rarely used model learns slowly); separate hosts cost money. A shared pool gives better average utilisation and worse tail behaviour. Start shared when load is small and the parts trust each other; add walls where a failure has already crossed one, or where the parts do not trust each other (tenants, models, test traffic versus the owner).

8. Defence in depth

8.1 Primer

Defence in depth layers independent controls so that one failure is not enough. James Reason's "Swiss cheese model" of accidents (upstream, 1990) is the usual picture: each layer has holes, and harm happens only when holes line up. The engineering discipline that follows is "assume breach": design each layer as if the one in front of it has already failed, and size its authority for that. The isolation review §4.1 states Choir's version as five principles (doc):

1. Assume each layer is breached; size the next layer for that.
2. One isolation contract, many runtimes; verify it from outside.
3. Privilege lives in small programs.
4. Authority is a credential, never a location.
5. Fail closed and sweep.

8.2 Choir incident: the VMM holds the keys

firecracker-runs-as-root-without-jailer (doc, live-read on Node B by the review): every Firecracker process ran as uid 0 with `CAP_NET_ADMIN` and `CAP_SYS_PTRACE`, no chroot, vmctl's mount namespace, and the host's network namespace. Firecracker's own seccomp filters were active, which is one layer. But the jailer, Firecracker's documented production launcher that gives each VMM its own uid, chroot, namespaces and cgroup, was not used; a search of `internal/vmmanager` and `internal/vmctl` for "jailer" finds nothing (code). So one guest-to-VMM escape that also bypasses seccomp lands as root with a view of every computer's data image. The review's verdict: "today boundaries B and D are thin, so A (the hardware plus Firecracker's code quality) carries almost all of the weight." That is one slice of cheese.

The second finding in the same review shows principle 4: "Loopback is a credential on Node B, and every host process holds it." vmctl's internal API, the gateway's credential routes and both Dolt servers trust any caller on 127.0.0.1, and the VMMs share that network namespace. A location ("came from loopback") was standing in for an identity.

8.3 How the layers should stack

```
model-written code        Yaegi cell: import allowlist (not a boundary)
      |                   capsule: namespaces, seccomp, Landlock, egress broker
      v
guest OS                  NixOS; guest root today
      |  boundary A       KVM + Firecracker device model
      v
VMM process               TODAY: root, host netns, shared mount ns
      |  boundary B       TARGET: per-VM uid, chroot, own ns, own cgroup
      v
host services             TODAY: loopback = trust
      |  boundary D       TARGET: per-service credential on every route
      v
keys and other tenants
```

Read it as a checklist: for each arrow, if everything above it is hostile, what does the layer below still refuse? Where the answer is "nothing", the layer is not a layer.

8.4 The question to ask next time

> *If the layer in front of this one is already compromised, what can the attacker reach from here, and what single check stops them?*

8.5 The trade-off

Each layer adds operational cost: jailed VMMs need per-VM files hard-linked or bind-mounted into a chroot, which complicates snapshots and balloon devices; per-service credentials need rotation; default-deny firewalls break things nobody listed. Layers also give a false sense of safety when they are not independent (two layers that share a misconfiguration are one layer). The review's rule of thumb is to put the next layer where the failure is most likely (the VMM parses attacker-controlled data at high speed) and to verify each layer from outside (`/proc/<pid>/status`), not from the code that set it.

9. Crash-only design and restart semantics

9.1 Primer

George Candea and Armando Fox's "Crash-Only Software" (HotOS 2003, upstream) argued that if a system must survive crashes anyway, the crash path should be the *only* shutdown path: stop by crashing, start by recovering. Then the recovery code runs every day and is trustworthy. The dangerous companion idea is automatic **resume**: a process that, on boot, picks up the work it was doing when it died. If that work is what killed it, the process crashes again, boots again, resumes again: a **crash loop**. Kubernetes's `CrashLoopBackOff` (upstream) exists because this is so common; backoff slows the loop but does not end it.

9.2 Choir's rule

CLAUDE.md, "Restarts End Work (Crash) Or Resume It (Planned Update)":

- **A crash restart never resumes work.** At boot every open obligation is closed with the visible fate "interrupted by a restart".
- **A planned restart may resume work**, once. The host writes a durable marker before the restart; the next boot consumes it before resuming. "A resume that crashes the computer leads to a markerless boot that closes the work: one resume per planned restart, so no loop."
- **Standing owner intents survive a crash; their occurrences do not.** A source schedule keeps polling after a crash; the poll that was in flight is closed.

The mechanism is `ConsumePlannedRestartMarker` (`internal/agentcore/planned_restart.go`, code). It removes the marker *before* reporting a planned boot; if removal fails, "treat this boot as a crash rather than risk a resume loop". A marker older than `plannedRestartMaxAge` (15 minutes), or from a different `CurrentWorkStateEpoch`, means crash. `RestartWorkKinds` lists the wake kinds that start or resume work; a pre-boot occurrence of one of them after a crash is `interrupted_by_restart`, never run. For standing intents, `recoverSourceSchedulesAtBoot` (`internal/agentcore/source_schedule.go`) closes a poll in flight, counts the crash against the schedule, and closes the schedule itself only at `sourceScheduleCrashLoopLimit = 2` crashes in a row: a bounded loop detector for the one kind of work allowed to continue.

9.3 Choir incident: the rule held for wakes, not for what they served

crash-restart-leaves-work-open-and-recasts-engineering (doc, from code) found two violations. First, a crash boot closed *actor wakes* of work kinds, but no boot path cancelled the *work items* behind them, and several boot paths (spawned-work replay, terminal settlement repair, Texture wake re-arming) then read that open work and acted on it: "the wake is closed, but the obligation it served stays open". Second, the engineering desk treated a restart's revocation of an assignment as system fate and **re-cast** it, up to three times, without checking the marker: "the crash-loop shape the rule forbids; the attempt cap only bounds the loop". The record also declined to patch: work items already had a bug cluster (five closure bugs in one day), so the direction was records-as-work, with the restart fate recorded on the record itself.

9.4 Choir incident: re-driving an apply, and why "mark it failed" was wrong

crash-boots-redrive-applies (doc) found that every boot, crash or not, re-pushed a pending platform update through `ApplyPlatformUpdate`, which called `updater.Apply` again. The obvious fix, "settle every crash-boot transition as failed", would have been false: if the updater's journal says `applied`, the new release is live; if the journal is mid-flight, nobody knows. The fix (18d01cc03, code) is `settlePendingPlatformUpdateAfterCrash`: it reads `updater.ReadJournalOutcome` and records exactly what the journal says (applied; failed with prior restored; or left visibly pending), and never calls the updater. This is the idempotency journal of section 4 doing a second job: it is also the crash-recovery source of truth.

The same record found a defect that shows how subtle markers are: `markPlannedRestart` cleared the marker only when the apply returned an error, so a resumed apply that succeeded without restarting left the marker on disk for up to 15 minutes, and "any crash in that window boots as planned".

9.5 The question to ask next time

> *If the process crashes in the middle of this, what does the next boot do with it: close it visibly, or pick it up? And if picking it up crashes again, what stops the loop?*

9.6 The trade-off

Closing work at every crash loses progress: a long research round, a half-applied update, an in-flight poll. That cost is real, and the owner's ruling on standing intents ("we need paper publishing 24/7, even after crashes") is a deliberate exception for work whose next occurrence is new work. The general shape: **resume intents, not occurrences**; when you do resume, make the resume single-use (consume the marker first) and bounded (a crash counter). The opposite choice, durable resumable workflows (upstream: Temporal, AWS Step Functions), is right when each step is idempotent and small, so a crash-inducing step can be identified and quarantined.

10. Backpressure and load shedding

10.1 Primer

A queue absorbs bursts. It cannot absorb a sustained excess. **Little's law** (upstream, John Little, 1961) says that in a stable system the average number of items in it equals the arrival rate times the average time each spends: L = λW. If arrivals λ exceed the service rate μ, there is no stable state: the queue and the wait grow without bound. Even below saturation, queueing theory says waiting time explodes as utilisation approaches 1 (for the simplest M/M/1 queue, mean time in system is 1/(μ − λ)). Systems survive overload with four tools:

- **bounded concurrency** (raise μ by running K workers, up to what the shared resources allow);
- **backpressure** (make the producer slow down when the consumer is full; TCP's receive window is the textbook case);
- **admission control** (refuse new work with a clear "retry later", HTTP 503 with `Retry-After`);
- **load shedding** (drop or degrade low-value work first).

And one anti-tool: **retries without backoff**, which raise λ exactly when μ fell.

10.2 Choir incident: one turn for everything

At 03:50 UTC on 14 October the owner asked "Is autopaper down?" (management-serial-turn-blocks-paper, doc). One management run had held management's only turn since 03:13. Management ran one turn at a time, and every kind of work (capture dispositions, Texture's requests, support judgement) went through that turn. The run was checking citations by hand instead of calling `choir.Judge`, on a free model whose per-call latency rose from 7 seconds to 9 minutes as its input grew from 7,704 to 113,382 tokens. Over a longer window, management was in a run 46% of the time and 105 of 132 capture arrivals found it busy. Nothing was dropped (per-poll limits are lossless); everything waited. A serial consumer with a heavy-tailed service time is a queue whose wait is set by its worst item.

The response, by owner direction ("we can't do serial"), moved support judgement to Jev and made management delegate bounded tasks to sub-RLMs (management-delegation-design, doc).

10.3 Choir incident: even one task at a time was too slow

After S2a (tasks, but one at a time), the paper went from 14 to 28 source schedules, and the backlog grew again (management-intake-backlog-at-28-sources, doc): 154 captures in 34 minutes, 5 disposed, 149 undisposed; 8 tasks completed, queue wait p95 1,506 s. The record estimates intake at about 270 captures an hour; 8 tasks in 34 minutes is about 14 tasks an hour. Unless each task disposes of about 19 captures, λ > μ and the backlog grows (arithmetic on the record's numbers; *inference* about per-task yield, which the record does not state).

S2b (433ce4d6e, code) raises μ with bounded concurrency: `managementTaskSlots` in `internal/agentcore/management_tasks.go` returns K (default 4, at most 8), "the rest queue, never refused". Read its comment for the three things that make concurrency safe here: "Concurrent tasks are kept coherent by the store, not by K: the lease and the disposition index (one capture, one disposition) and the story-library head". It also has a live rollback: the owner's `management.serial_tasks` setting sets K = 1, "read on every start, so it needs no restart". And there is a shared resource that K does not raise: the embedded Dolt engine is serialized by `engineMu` (`internal/objectgraph/dolt_store.go`, code), so the design lists "H3: engine-lock waits stay small at K = 4" as a hypothesis to measure. Concurrency moves the bottleneck; it does not remove it.

10.4 Choir's admission and backoff tools

- **Admission control:** vmctl refuses a new unprotected computer with `host_memory_pressure` and `RetryAfterSeconds` (section 7.4).
- **Rate limit:** the gateway allows 60 inference requests a minute per computer (`DefaultRateLimitMaxRequests`, `internal/gateway/ratelimit.go`, code).

- **Bounded retry with backoff:** `scheduleFailedTurnRetry` (section 4.3) doubles the delay per failed turn and stops after four.

10.5 The question to ask next time

> *What is the arrival rate, what is the service rate, and what happens to the queue when the second drops below the first for an hour?*

10.6 The trade-off

Raising K raises throughput and contention: more model calls against a per-computer rate limit, more commits against one engine lock, more memory (about 17 MiB per desk worker, per the design). Admission control keeps the system healthy by making some users wait or fail, which is a product decision, not only a technical one; Choir's owner has ruled that bounds may limit concurrency but never cap model spend (design §7, doc). Load shedding needs a value order (which captures matter less?), and choosing it badly drops the story that mattered. A queue that "never refuses" is honest only if someone watches its age.

11. Observability, and reading the trace before attributing behaviour

11.1 Primer

Observability is the ability to answer new questions about a running system from its outputs (logs, traces, metrics, profiles) without shipping new code. The deeper discipline is epistemic: **absence of an effect is not evidence of a cause**. A request that was never sent, one that was sent and dropped, and one that was answered late all produce the same missing downstream effect. Only a record of what actually happened, in order, separates them. CLAUDE.md states it for Choir: "Review the traces before attributing behavior. [...] An instruction that *was* followed but committed late looks identical to one that was never heeded — only the trace distinguishes them. A behavior claim made without trace evidence is a hypothesis, not a finding."

Its companion rule is "Problem Documentation First": the first commit after evidence of a new problem documents it, without a fix, so the problem can be reviewed separately from any one solution (memo, doc). The memo's receipt: a commit that serialized heavy tool calls after a CPU-saturated VM, authored in the same context that first saw the problem, so alternatives (per-tool CPU budgets, cgroups, bounded concurrency) were never weighed.

11.2 Choir incident: the CPU was not the agents

The paper's computer used about 1.3 cores. The natural guess was agent work: twelve Yaegi workers, model calls, JSON. A 60-second guest CPU profile (guest-cpu-is-unindexed-store-scans, doc, with the raw profile in `docs/evidence/computer-cpu-guest-profile-2026-10-14T1646Z.txt`) showed the main daemon at 1.17 cores (98% of guest time) and every other guest process, the Yaegi workers included, at 0.03. 73% of the daemon's CPU was under `database/sql.(*Rows).Next` in the embedded Dolt engine, and 52% was one path: listing inbox records by filtering `JSON_EXTRACT(metadata, ?) = ?` over every `commitment_record`, at the end of every desk run. The cost scales with the number of stored records, not with the work done. An indexed half-replacement already existed (`metadataIndexColumns`, generated columns in `internal/objectgraph/dolt_store.go`, code) but was not used by the paged read. Without the profile, the obvious "fix" would have been to reduce agent work, which would have changed nothing. (Part 1 teaches the indexing.)

11.3 Choir incident: an error the trace never recorded

desk-verb-error-vanishes-as-final-expression (doc): a research desk ended a cell with a bare `choir.ReportPacket(...)`. The verb returned `(string, error)`; the interpreter shows only the first value, so the cell result was `=> ` with `error ""`. The packet had been refused, and a round's worth of funding leads never reached the document. Two cells in 721 did this, each silently losing a round. The fix (4ca9b0947, code) collects every verb refusal during a cell and shows it in the cell's result (`refusals []verbRefusal` in `internal/`

`yaegikernel/choir.go`): "so a refusal is never lost when the cell ends in a bare call". The principle: **an error that is not recorded where the decision-maker reads did not happen**, for every practical purpose. The same lesson appears in the 400-breaker record: providers discard non-2xx bodies (they may contain credentials), so the reason for the nine 400s "is not recorded anywhere". The fix logs 4xx reasons, redacted and truncated (589a68bc6).

11.4 Choir incident: reading the trace instead of guessing intent

The management serial-turn record (section 10.2) is a model of the method. It did not say "management is lazy" or "the model ignored the prompt". It read the run's events in order: iteration 1 listed records; iterations 2 and 3 were compile errors guessing field names; iterations 4 to 11 read captures one or two per cell; it never called `choir.Judge`; latencies per call; input tokens per call. Then it named the substrate cause (one serial turn for all work; a prompt that offered "judge yourself" as an option) and labelled the generalisation ("management often judges by hand") as a hypothesis, not yet confirmed across runs.

11.5 The question to ask next time

> *What does the trace say happened, in order? Which part of my explanation is read from it, and which part am I inferring?*

11.6 The trade-off

Recording everything is expensive (storage, CPU, privacy) and can leak secrets; Choir's providers deliberately discard error bodies for that reason, and the fix logs a redacted, truncated reason instead. Traces must be designed: record decisions and their inputs, errors with their cause, and timings at boundaries, not every byte. And documentation first has a cost in hours; CLAUDE.md allows an exception only "with explicit justification in the commit message".

12. Pre-flight card

Ask these of any design change in Choir. Each maps to a principle and to the incident that taught it.

#	question	principle	incident that taught it
1	If this is alive but silent, who notices, and how fast?	failure modes, gray failure	sigpipe deadlock; vmctl health fate-shares a hung guest
2	Where does model output become state or reader text, and what validates it there?	Byzantine components	instruction text as titles
3	Does every timeout exceed the whole budget of what it waits on?	deadline budgets	search timeout equals gateway timeout
4	Does every wait have a host-enforced bound, and does a bad config keep the bound?	deadline budgets	management cell ran 14 minutes
5	If two of these run at once, how does each know its answer is its own?	correlation, races	parallel calls cross-wire; observer race
6	Delivered twice: what happens? Dies right after acknowledging: what is lost?	idempotency, ack after work	captures undisposed after failed runs
7	Who is the one writer of this fact, and can any other path change it?	single state authority	instruction text as titles; standing question 5
8	When this check cannot decide, does it allow or deny, was that chosen, and what bounds it?	fail closed / open	tap deny covers 1 of 16; owner computer stranded
9	Does this breaker or guard open only on evidence about the thing it guards?	fail closed, blast radius	breaker trips on client errors; free-model 429
10	If this slows or fails, what else fails with it, and is each on purpose?	blast radius, bulkheads	deploy drain takes site down; Node B OOM
11	If the layer in front is breached, what can the attacker reach from here?	defence in depth	Firecracker as root without jailer

#	question	principle	incident that taught it
12	If it crashes mid-way, does the next boot close it or resume it, and what stops a loop?	crash-only, restart semantics	crash restart leaves work open; crash boots re-drive applies
13	Arrival rate versus service rate: what happens to the queue for an hour of overload?	backpressure	serial turn; backlog at 28 sources
14	Is my explanation read from the trace, or inferred? Is the problem documented before the fix?	observability, documentation first	guest CPU is store scans; verb error vanishes

Two meta-questions from CLAUDE.md "Convergence Before Patching" sit above the card:

- **Substrate or symptom?** If three or more bugs land in the same subsystem in a week, stop patching and write a clustering assessment. The records above do this repeatedly: the work-item closure cluster pushed restart fates onto records; the CPU record named three symptoms of "the object graph used as a JSON document store with scan-and-filter reads under one engine mutex".
- **Is there an existing replacement?** The embedded Dolt single connection (problem record, doc) worked around a data race that upstream Dolt had already fixed in commit 6bea93f2. `go.mod` now pins `github.com/dolthub/dolt/go v0.40.5-0.20260716231223-6bea93f2f665` (code), past the fix, while `engineMu` still serializes the engine (code). The record's plan was to remove the mutex "in measured steps" only after race tests pass; this guide did not check where that stands (*inference*: the upgrade is a precondition, not the removal).

Glossary

term	meaning
Acknowledgement (ack)	Telling a sender or a queue that a message is handled, so it will not be redelivered. Safe only after the work is durable, or in the same transaction.
Admission control	Refusing new work at the door when the system is near capacity, with a clear retry signal (HTTP 503 and `Retry-After`).
At-least-once / at-most-once	Delivery guarantees: retry until acknowledged (may duplicate), or send once (may lose).
Backpressure	A signal from a full consumer that makes the producer slow down.
Blast radius	What fails when one thing fails.
Bulkhead	A wall that limits blast radius: a separate pool, process, cgroup, breaker or host.
Byzantine failure	A part that answers wrongly, possibly plausibly. A language model is one by nature.
Circuit breaker	A guard that stops calling a failing dependency for a while and fails fast. Correct only if it counts the right failures and is keyed at the right grain.
Compare-and-set (CAS)	A write that succeeds only if the current value is the one the writer expected (`ExpectedHeadRevisionID`, `ExpectedSeq`). The basis of optimistic concurrency.
Crash loop	Boot, resume the work that crashed, crash again.
Crash-only software	Design where crash and recovery are the normal stop and start paths.
Deadline propagation	Passing the caller's absolute deadline down the call chain, so every layer stops when the caller does.
Defence in depth	Independent layers, each sized for the failure of the one in front.
Dual-write problem	Writing one fact to two stores without a transaction; a crash between the writes leaves them disagreeing.
Fail closed / fail open	Deny or allow when a check cannot decide.
Fate-sharing	Two parts that fail together because one depends on the other, often without anyone choosing it.
Gray failure	A part that its health check considers healthy while its users experience it as broken.
Idempotency key	A name for an operation that survives retries; the handler records the outcome under it and returns that outcome on repeats.
Little's law	L = λW: items in a stable system equal arrival rate times time in system.
Load shedding	Dropping or degrading low-value work under overload.

term	meaning
Planned-restart marker	Choir's durable file, written before an intentional restart and consumed once at the next boot; without it a boot is a crash.
Single-writer principle	Every fact has one writer; every copy is derived.
Standing intent	An owner instruction that outlives occurrences (a source schedule); it survives a crash, its in-flight occurrence does not.
Swiss cheese model	Reason's picture of accidents as aligned holes in independent layers.

Self-check

1. The guest's search client and the gateway's search handler both used a 30-second timeout. Explain precisely why that produced failures that "did not happen", and state the nesting rule that prevents it. Why did Choir end up removing the synchronous wait instead?
2. In the 08:22 burst, 28 captures were lost although every run that touched them either failed visibly or never ran. Which line of reasoning in `commitRecordsRoute` made reading equal to acknowledging, and what is the general rule about where an acknowledgement belongs?
3. `ConsumePlannedRestartMarker` treats an unreadable marker as a crash, while `admitHostMemoryLocked` admits a computer when its memory sample is missing. One fails closed and one fails open. Justify each by naming which wrong answer is worse and who pays for it.
4. A free model's 429s opened a breaker that blocked a healthy paid model. Name the two properties of a circuit breaker that were wrong across this incident and its 10 October predecessor, and say how the current `CircuitBreakingProvider` handles each.
5. Management ran tasks one at a time and fell behind at 28 sources. Using Little's law, explain why a longer queue would not have helped, and name one shared resource that raising K to 4 does not raise.

Answer sketches

1. The gateway's work could take its full 30 s and succeed after the guest, also at 30 s, had already returned a timeout to the cell; with fallbacks the gateway could exceed 30 s by design. Rule: each caller's timeout must exceed the whole budget (including retries and fallbacks) of what it waits on, ideally by propagating a deadline. Async search removed the caller's need to wait for the slowest provider at all, so the tie stopped mattering.
2. A cell that staged no acts still committed a cursor advance over what `choir.Records()` had shown, so a read-only cell marked records read; the run then failed before disposing of them. An acknowledgement belongs after the work is durably done, or in the same transaction as the work; the fix marks read at the end of a completed or parked turn.
3. Marker: a wrong "planned" resumes work and can crash-loop the computer; a wrong "crash" closes work visibly, which the owner can restart. Memory: a wrong "refuse" (broken sampler) stops every boot on the host; a wrong "admit" risks pressure that reconcile, reclaim and OOM priorities still bound. In both cases the cheaper wrong answer was chosen.
4. What counts: 4xx client errors and caller cancellations counted as provider failures (fixed by `providerFailureCounts`). Key grain: one breaker per provider for all its models (fixed by keying per (provider, model) plus a provider breaker that counts only `providerUnreachable` transport failures).
5. With arrivals near 270 an hour and roughly 14 tasks an hour completed, the system had no stable state unless each task disposed of many captures; a longer queue only lengthens the wait ($W = L/\lambda$ grows without bound when $\lambda > \mu$). Raising K raises μ, but the embedded Dolt engine is still serialized by `engineMu`, and the gateway still allows 60 inference calls a minute per computer.

Read these three files

1. **`internal/agentcore/planned_restart.go`.** The whole crash-versus- planned rule in about four hundred lines. Read `ConsumePlannedRestartMarker` for how every doubt resolves to "crash", and the comment on `RestartWorkKinds` for which work counts as resuming.
2. **`internal/gateway/circuit_breaker.go`.** A small, heavily commented example of failure classification and blast-radius keying. Read `providerFailureCounts` and `providerUnreachable` side by side, and the type comment on `CircuitBreakingProvider` for why there are two grains of breaker.
3. **`docs/problems/captures-undisposed-after-failed-management-runs-2026-10-14.md`**, then `markTurnRecordsRead` in `internal/agentcore/inbox_records.go`. The best end-to-end example in the repository of a trace read in order, a mechanism confirmed in code, a fix with idempotent, CAS-guarded, bounded-retry acknowledgement, and an honest remaining error field.

Sources and confidence

- **Read in code at 980ffaf4e (each path verified to exist):** `internal/yaegikernel/worker_mux.go`, `internal/yaegikernel/choir.go`, `internal/computerevent/appender.go`, `internal/updater/updater.go`, `internal/agentcore/platform_update.go`, `internal/agentcore/planned_restart.go`, `internal/agentcore/source_schedule.go`, `internal/agentcore/inbox_records.go`, `internal/agentcore/management_tasks.go`, `internal/agentcore/tools_desk.go`, `internal/agentcore/story_publish.go`, `internal/store/inbox.go`, `internal/store/story_open.go`, `internal/store/texture_turn.go`, `internal/texturedoc/headline.go`, `internal/textureowner/texture_cell_author.go`, `internal/gateway/circuit_breaker.go`, `internal/gateway/ratelimit.go`, `internal/gateway/handlers.go`, `internal/gateway/search_plane.go`, `internal/search/search.go`, `internal/toolregistry/toolloop.go`, `internal/vmctl/host_capacity.go`, `internal/vmmanager/manager.go`, `internal/objectgraph/dolt_store.go`, `internal/vmctl/wedge.go`, `nix/node-b.nix`, `go.mod`. Fix commits confirmed as ancestors of this branch: cd2a224f5, 19e56c54e, 18d01cc03, 589a68bc6, bde0a169f, e3437f1cd, 433ce4d6e, 2db4bc02, cf0969cf, 86770ae7, 201fe33ca, f10b655a4, 4ca9b0947, 7f287a135, 88734e8ab. The tap deny fix 816077e83 is on branch `fix/tap-deny-guest-range` only.
- **Read in problem records (doc), which cite live traces this guide did not re-read:** every incident narrative and every number in sections 1 to 11 (latencies, counts, timestamps, live host state). Where a record labels something a hypothesis, this guide keeps the label.
- **Inference, marked in the text:** that unknown capture ids are still accepted by `choir.Feed`; that S2b's disposition index is the right shape for the partial-turn residual; the twelve-attempt retry upper bound; the per-task yield needed to keep up at 28 sources; the status of removing `engineMu` after the Dolt upgrade.
- **Upstream ideas named, not quoted:** Lamport, Shostak and Pease (1982); Huang et al., "Gray Failure" (HotOS 2017); Dean and Barroso, "The Tail at Scale" (CACM 2013); the Two Generals Problem; Stripe's idempotent requests; the transactional outbox; Thompson's single-writer principle; Nygard, *Release It!* (2007); Reason's Swiss cheese model (1990); Candea and Fox, "Crash-Only Software" (HotOS 2003); Little's law (1961); Amazon Builders' Library on cells and shuffle sharding; Kettle's request-smuggling research (PortSwigger).
- **Not covered:** consensus and replication (Raft, Paxos), which Choir does not run today; clock skew and ordering across hosts; the security of auth sessions and tokens. See the series map for what to learn next.
- **Sibling parts:** ledger and store, desk runtime, gateway and providers, computer lifecycle, publication and Autopaper.